Sicura

PROJECT FOR EMBEDDED SOFTWARE FOR INTERNET OF THINGS

SW/HW Architecture

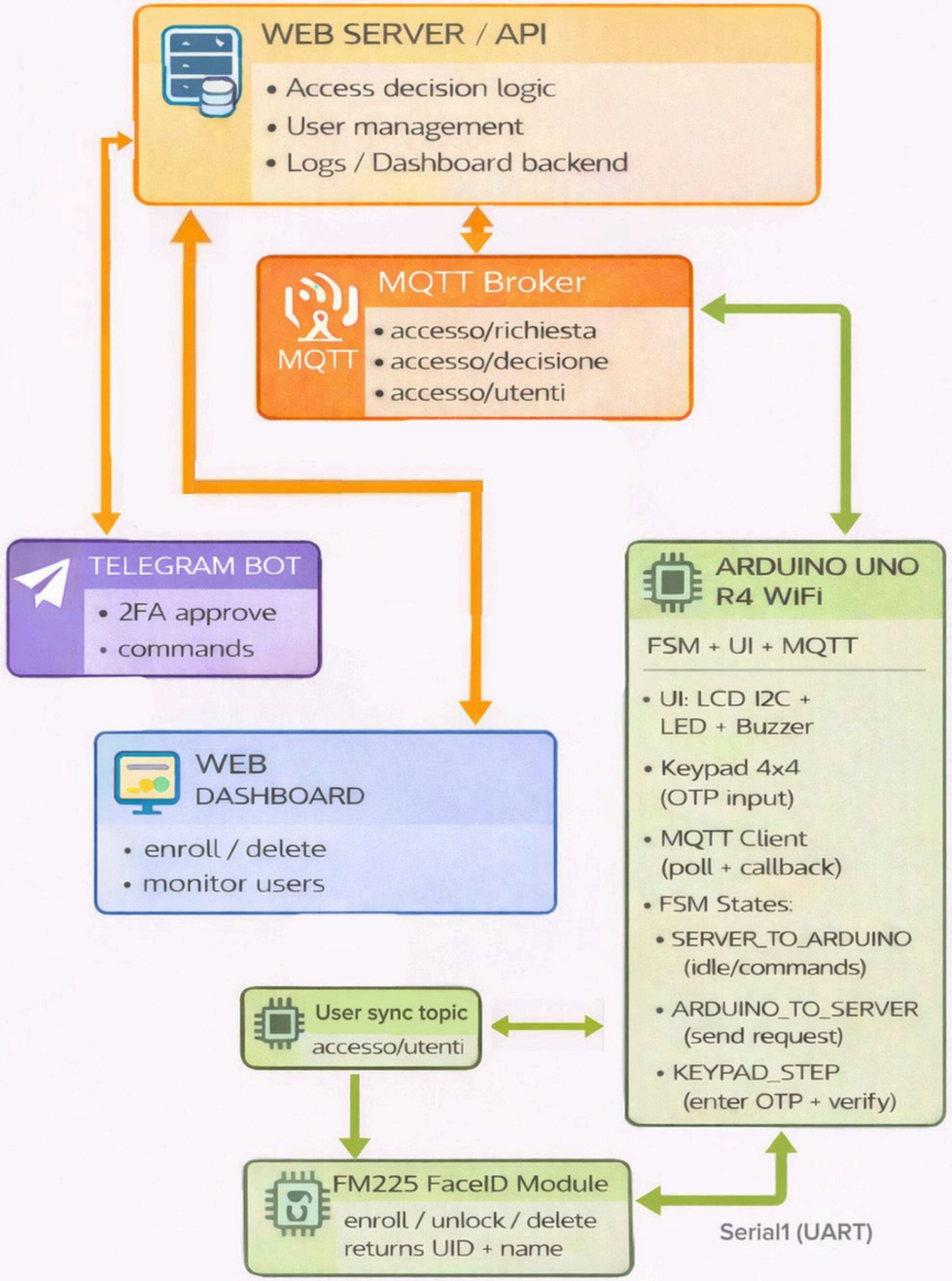
Interrupts / Event Handling
- Only interrupt: Buzzer timer ISR (FspTimer, 1 ms callback, timer_callback)
- ISR: advances beep sequence

No attachInterrupt, no GPIO/UART ISRs → everything else polling

Sensing / Processing
- MQTT: mqttClient.poll() → messageReceived()
  Topics: accesso/decisione (OK/KO + OTP), accesso/utenti (ENROLL/DELETE + results)
- FaceID (UART): Serial1.available() → readFM225() → parseReply/parseNote
- Keypad: Adafruit_Keypad.tick() + available() → PIN entry / actions

Data Structures / Algorithms
- FSM: SERVER_TO_ARDUINO → ARDUINO_TO_SERVER → KEYPAD_STEP
- User table: UserEntry userTable[32] (uid, name, valid, isAdmin)
- Logic: MQTT pub/sub, OTP PIN match, FaceID frame parsing

Server objects
Admin/User model:
- id, (with prefix “usr” or “adm”)
- username,
- passwordHash (admin),
- telegramUsername,
- telegramChatId,
- enabled,
- faceRegistered (user)
- sessionId (admin).



FaceID (FM225)

Serial Packet Structure / Framed Protocol
[0] 0xEF
[1] 0xAA → SyncWord (start packet)
[2] msgID → actions
[3] sizeH → most significant byte (big-endian)
[4] sizeL → less significant byte (big-endian)
[...] payload → size = (sizeH<< 8) | sizeH; the first byte is the noteID
[last] checksum (parity)

```c
uint8_t parity(uint8_t *packet, int len) {
  uint8_t p = 0;
  for (int i = 2; i < len; i++)   // XOR
    p ^= packet[i];
  return p;
}
```

action	payload size
UNLOCK	2 byte
DELETE	2 byte
ENROLL	35 byte

msgID	action
0x02	Ping / Status
0x12	UNLOCK
0x1D	ENROLL
0x20	DELETE USER
0x24	GET USERS

MQTT/ ISR

```
if (!buzzerUpdate) return;
noInterrupts(); bool active = buzzerActive;
int f = buzzerFreq; buzzerUpdate = false;
interrupts();
if (!active || f <= 0) noTone(A3);
else tone(A3, f);
```

Interrupt-safe buzzer update: ISR sets state, main loop applies tone/noTone atomically to avoid race conditions.”

MQTT request/response: Arduino publishes an access request (JSON) on accesso/richiesta; the server subscribes, decides (admin/Telegram), then publishes decision + OTP on accesso/decisione.

```
mqttClient.beginMessage(topic_richiesta);
mqttClient.print(json);
mqttClient.endMessage();
```

```
const msg=JSON.stringify(payload);
client.publish("accesso/decisione",msg);
```

```
client.subscribe("accesso/richiesta");
client.on("message",(t,m)=>handleMessage(t,m.toString()));
```

Several Drawbacks

- **MQTT polling**: handled via the Connected status (connection-state management).
- **Keypad debounce issue**: the keypad library's pull-up handling wasn't compatible with the R4 WiFi (Renesas ARM Cortex-M4) compared to older AVR-based Arduinos; fixed by switching to a different keypad library.
- **LCD flickering**: caused by moving from a parallel to a serial interface, introducing timing/clock-related artifacts.
- **FM225 module**: lack of community support and poor documentation; only partial hex frame interpretations available, and frames vary across hardware revisions.
- **FaceID UART framing**: added packet framing (length field + checksum) and a retry mechanism to improve reliability.

AVR microcontrollers produced by Atmel
arduino ARM Cortex-M4 microcontrollers produced by Renesas

Future Additional Features and Conclusions

- **Multi-admin** support & user partitioning
- **Hardware hardening**: custom PCB (replace breadboard & jumpers)
- **Protection** against physical tampering / firmware replacement
- Security camera on unknown requests
- Robust **database** backend (replace JSON storage)
- Real TLS for MQTT (**mqtts**)
- Secure **secrets management** (JWT, TEL_TOKEN)

THANK YOU

Anas Soussane
anas.soussane@studenti.unitn.it
number of code rows written: 3780 (more or less)
+ 1300 firewall

Andrea Dalla Villa (ID242637)
andrea.dallavilla@studenti.unitn.it
number of code rows written: 1450 (more or less)